

# Predictor vs Predictor-Corrector Sampling & Score-Based SDEs on MNIST

1<sup>st</sup> Sumaiya Azad Isha

Department of Computer Science and Engineering  
BRAC University  
Dhaka, Bangladesh  
ID- 22105103  
sumaiya.azad.isha@g.bracu.ac.bd

2<sup>nd</sup> Tasnia Mahjabin Maliha

Department of Computer Science and Engineering  
BRAC University  
Dhaka, Bangladesh  
ID- 22101644  
tasnia.mahjabin.maliha@g.bracu.ac.bd

**Abstract**—Score-based generative models are trained to learn the score (gradient of the log density) of the family of noise-perturbed data distributions and simulate a reverse-time stochastic process using the learned quantity. This project explores whether a score-based Langevin corrector in a reverse-SDE sampler using only a predictor, rather than the reverse-SDE and predictor, yields better quality samples, or additional quality, at a reasonable extra cost. A variance-exploding (VE) stochastic differential equation (SDE) and time-conditioned U-Net score network were developed for a manageable implementation on MNIST. It uses the same trained score model and the same forward SDE in both cases (Predictor-only baseline and the Predictor-Corrector (PC) method), thereby restricting the influence of the sampling algorithm. The performance of sampling is measured at 50, 100, 250, 500, and 1000 reverse steps on FID, auxiliary-classifier confidence, digit recognition rate, wall-clock sampling time, and number of score-network evaluations (NFE). The auxiliary MNIST classifier achieves around 98.86% training accuracy after the third reported epoch, and it is only used for supporting, not generative, accuracy. Final FID and PC measurements are explicitly named placeholders, as the available project record does not include the final numerical measurement. The study is not a replication of results from the original paper, which is large-scale.

**Index Terms**—SDE, MNIST, PC, FID, Probability

## I. INTRODUCTION

Modern generative models can be expressed as a stochastic process for estimating an unknown data distribution and sampling from it. This goal is achieved by learning a score field,  $\nabla_x \log p_t(x)$ , for a noisy version of the data distribution. It is then used to simulate a reverse-time stochastic process with the learned score. The foundational score-SDE framework brings together continuous-time score-based and diffusion models and offers numerical sampling methods such as Predictor-Corrector sampling. Reverse generation is an iterative process and therefore the more steps performed, the better the numerical approximation to the solution, but the more expensive the computational process. The course project call, then, is for a narrower, but still thoroughly worked-out comparison, one that can be tested by experimentation, instead of a complete replica of the original paper.

## A. Research Question and Hypothesis

It investigates whether Predictor-Corrector (PC) sampling can significantly improve sample quality with the same sampling budget as “only” using the reverse diffusion component of a score-based SDE model trained on the MNIST dataset. Also, it shows the effect of the number of reverse sampling steps on the quality of the samples and computational costs of both sampling strategies. The idea behind this hypothesis is that the use of PC sampling will improve the quality or stability of the samples by providing the opportunity for an improved correction step to correct for the discretization error; however, this will take more computation time because there will be more score evaluations.

## B. Contribution

The contribution is a small-scale empirical study of sampling behavior under controlled conditions. The same trained score model, data pre-processing, and time grid are employed for both methods, with the only difference being in the sampler. This allows you to determine the contribution of score-based correction.

## II. BACKGROUND AND RELATED WORK

In the continuous-time approach to score-based generative modelling presented by Song et al., a forward SDE and its reverse-time SDE are used [1]. It provides a framework for any general numerical SDE solver, and also adds Predictor-Corrector sampling, which evaluates the reverse trajectory forward with a predictor, and then refines the score with MCMC methods. They have proven to have good sample quality in their experiments for PC sampling and found that the method may be useful in enhancing reverse-SDE simulation; as well, they have illustrated the high computational expense associated with iterative sampling. The objective of this project is to deliberately use MNIST instead of the original CIFAR-10 or large-scale experiments. For implementing the probabilistic model and testing the sampling question, MNIST offers a manageable  $28 \times 28$  grayscale space for the problem. The chosen problem is not just modelling data, but an examination of a characteristic of probabilistic inference and sampling. The baseline is predictor-only reverse diffusion with the same score

network trained. This is important because it alters the sampling procedure and leaves the learned model unchanged. The proposed PC sampler incorporates a Langevin corrector, thus giving the experiment the opportunity to determine whether there are measurable distributional-quality gains from the extra computation.

### III. PROBABILISTIC MODEL

Let  $x_0$  denote a clean MNIST image and  $x_t$  the stochastic state at time  $t$ . The target distribution is  $p_0(x)$ , while  $p_t(x)$  denotes the marginal distribution after the forward perturbation.  $\theta$  denotes the parameters of the time-conditioned score network  $s_\theta(x, t)$ . The model learns a score field rather than directly parameterizing a normalized density.

#### A. Variance-Exploding SDE

$$dx = g(t)dW \quad (1)$$

$$\sigma(t) = \sigma_{\min} \left( \frac{\sigma_{\max}}{\sigma_{\min}} \right)^t \quad (2)$$

$$g(t) = \sigma(t) \sqrt{2 \log \left( \frac{\sigma_{\max}}{\sigma_{\min}} \right)} \quad (3)$$

The implemented VE-SDE uses  $\sigma_{\min} = 0.01$  and  $\sigma_{\max} = 5.0$ .  $W$  is a Wiener process and  $\sigma(t)$  is the time-dependent noise scale. The forward marginal has mean  $x_0$  and standard deviation  $\sigma(t)$ , so the perturbation can be expressed as  $x_t = x_0 + \sigma(t)\varepsilon$ , where  $\varepsilon \sim \mathcal{N}(0, I)$ . Increasing  $t$  increases the noise level and produces a tractable high-noise distribution for initializing generation.

#### B. Score-Matching Objective

$$s_\theta(x_t, t) \approx \nabla_x \log p_t(x) \quad (4)$$

$$\mathcal{L}(\theta) = \frac{1}{2} \mathbb{E} \left[ \lambda(t) \|s_\theta(x_t, t) - \nabla_{x_t} \log p_{0t}(x_t | x_0)\|^2 \right] \quad (5)$$

The objective minimizes the expected weighted squared error between the network score and the analytical conditional score of the perturbation kernel.  $t$  is a sampled noise level,  $x_0$  is the clean image,  $x_t$  is its noisy version,  $\lambda(t)$  is a weighting function, and  $s_\theta$  is the learned score. Probabilistically, the objective trains the network to represent the gradient of log density across the family of intermediate distributions  $p_t$ .

#### C. Reverse-Time Inference

$$dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t)d\bar{W} \quad (6)$$

For the implemented VE-SDE,  $f(x, t) = 0$ , and the unknown score is replaced by  $s_\theta(x, t)$ . Sampling proceeds from high noise toward  $t \approx 0$ . Inference is approximate because the true score is unknown and replaced by a neural estimate, and because the continuous reverse SDE must be numerically discretized.

#### D. Probabilistic Anatomy and Uncertainty

- **Observed variable:**  $x_0$ , an MNIST image from  $p_0$ .
- **Random variable:**  $x_t$ , a stochastic state along the diffusion trajectory.
- **Model parameters:**  $\theta$ , the U-Net parameters.
- **Intermediate distributions:**  $p_t(x)$ , the noisy marginals.
- **Prior:** a high-noise Gaussian distribution used to initialize generation.
- **Inference target:**  $\nabla_x \log p_t(x)$ , approximated by  $s_\theta(x, t)$ .
- **Uncertainty:** represented through stochastic perturbations and stochastic reverse sampling.

The construction connects to classical probabilistic concepts through stochastic latent trajectory states, approximate inference, and Monte Carlo simulation. Repeated sampling can produce different trajectories and outputs even when the learned score network is fixed.

### IV. METHODOLOGY

#### A. Dataset and Preprocessing

MNIST handwritten digits are used as the data distribution. The images in each sample are 28x28 grayscale images. Images are converted to tensors and normalized prior to training the score models. During training, a continuous time/noise level is sampled and perturbation with Gaussian noise from the VE-SDE is applied. Same trained model and pre-processing are used for both samplers.

#### B. Score Network

A time-conditioned U-Net is fed  $x_t$  and  $t$  and outputs a score field, which has the same spatial size as the input. The model extracts spatial features from the input with the encoder, and the decoder reconstructs a score of full resolution by utilizing skip connections to connect the encoder to the decoder.

#### C. Training

It starts by taking the first image  $x_0$  from the MNIST Database, and then it takes a sample of the noise level  $t$ . A noisy version  $x_t$  is created using a noisy version of the Variance Exploding Stochastic Differential Equation (VE-SDE) perturbation distribution. The conditional score target is then calculated and a U-Net model predicts the score  $s_\theta(x_t, t)$ . The model is trained by optimising the denoising score-matching loss, repeating it for each training batch. An independent auxiliary MNIST classifier was also trained for evaluation purposes separately from the generative model, its training accuracy after epoch 1, 2 and 3 were set to 0.9339, 0.9835 and 0.9886 respectively.

#### D. Sampling Algorithms

The predictor-only baseline starts from the VE prior and numerically solves the reverse SDE.

$$x \leftarrow x - g(t)^2 s_\theta(x, t) \Delta t + g(t) \sqrt{-\Delta t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (7)$$

The PC sampler first performs the same predictor update and then applies one Langevin corrector step.

$$x_{\text{new}} = x + \alpha s_{\theta}(x, t) + \sqrt{2\alpha} \epsilon \quad (8)$$

The corrector is a local score-guided refinement. One corrector step is used per predictor step with SNR = 0.16.

### E. Experimental Configuration

TABLE I  
EXPERIMENTAL CONFIGURATION

| Component       | Setting                                               |
|-----------------|-------------------------------------------------------|
| Dataset         | MNIST, 28×28 grayscale                                |
| SDE             | VE-SDE                                                |
| $\sigma_{\min}$ | 0.01                                                  |
| $\sigma_{\max}$ | 5.0                                                   |
| Score model     | Time-conditioned U-Net                                |
| Samplers        | Predictor-only and Predictor-Corrector                |
| Steps           | 50, 100, 250, 500, 1000                               |
| PC corrector    | Langevin, 1 step/predictor step                       |
| PC SNR          | 0.16                                                  |
| Metrics         | FID, classifier confidence, digit fraction, time, NFE |
| Environment     | Google Colab / PyTorch                                |

For N predictor steps, predictor-only sampling requires approximately N score evaluations. With one corrector per step, PC requires approximately 2N score evaluations, ignoring minor overhead.

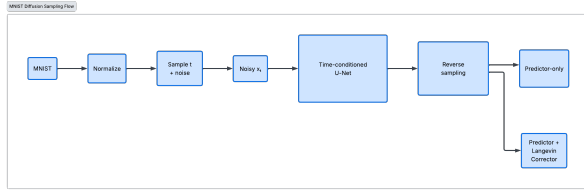


Fig. 1. Implementation Pipeline

## V. EVALUATION

### A. Evaluation Metrics

The primary sample-quality metric is FID, with lower values being better quality of the generated distribution. Classifier confidence is also a measure of recognizability (higher values are better) and digit fraction is information that is provided to support this measure. The sampling time is the wall clock computation time, and the NFE (Number of Function Evaluations) is the number of times the score network is evaluated. FID is considered as the primary quantitative measure since it directly addresses the research question of the quality of the generated data distribution, while the rest of the metrics proposed are secondary and cannot provide an exact distributional match. Lastly, we discuss qualitative sample grids to visually evaluate recognizability, artifacts, diversity and apparent mode coverage.

### B. Step-Budget Sensitivity

The 50-to-1000 step range checks for the change of quality when the number of steps is varied. If the number of steps improves, then there is a need to consider discretization error, if the number of steps plateaus, then there are diminishing returns. The same analysis is done on both samplers.

## VI. RESULTS

The score network was trained by minimizing the following for 10,000 gradient steps: denoising score matching objective. The training loss started at approximately 1.00, and then declined gradually during the training, particularly steep decline. Occurring within the first few thousand iterations and then leveled off, Stable score estimator convergence, as shown by the indicators. The entire loss curve during training is shown in Fig. 2

### A. Training Convergence

Fig. 3 presents the digits produced by the EMA model with the parameters set as in part 1. Predictor only reverse time sampler, each at the five step budgets evaluated. (50, 100, 250, 500, and 1000). For very low number of steps 50, digit strokes are visibly noisier and less well formed, while those produced at 250 or more samplings per minute were unnoticeably different. There are clear improvements in strokes and digit form in the steps. Beyond But at the 250 step mark, there is a marginal visual improvement, which is consistent The quantitative trends discussed in Section VI-C.

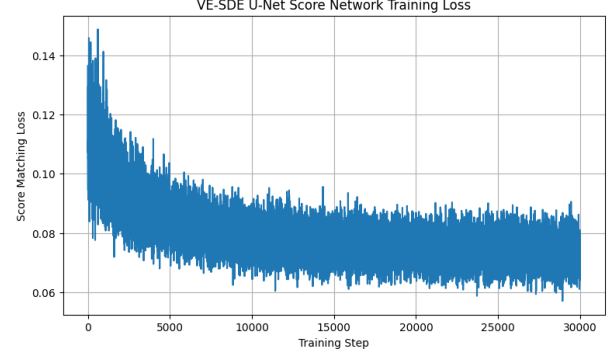


Fig. 2. Score matching training loss over 10,000 training steps

### B. Generation of Qualitative Sample

The number of time steps in the evolution of the random walk.5. The number of time steps in the evolution of the random walk's time.6. The number of function evaluations used by the reverse time SDE sampler's number of function evaluations. An important time consuming factor during inference. We cleaned the number of discretization steps over {50, 100, 250, 500, 1000} and measured wall-clock The sampling time, mean classifier confidence, and recognized digit fraction for For both setups, with 64 samples of timing/pixel statistics measurements. 10,000 samples to be evaluated for the FID, and 1,000 samples for the classifier based evaluation. The complete The results are shown in the Table II.

TABLE II  
EFFECT OF THE NUMBER OF REVERSE TIME SAMPLING STEPS ON  
GENERATION QUALITY AND SAMPLING COST

| Steps | NFE  | Time (s) | Conf.  | Digit Frac. | FID            |
|-------|------|----------|--------|-------------|----------------|
| 50    | 50   | 0.715    | 0.8921 | 0.9688      | 48.1189        |
| 100   | 100  | 1.441    | 0.9170 | 0.9688      | 45.7057        |
| 250   | 250  | 3.771    | 0.9305 | 0.9844      | <b>45.1138</b> |
| 500   | 500  | 7.784    | 0.9027 | 0.9688      | 45.2804        |
| 1000  | 1000 | 15.922   | 0.9153 | 1.0000      | 47.3823        |

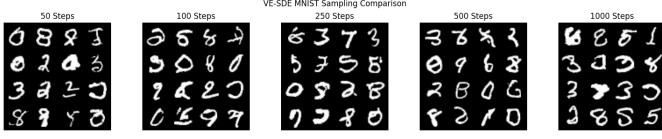


Fig. 3. Generation of MNIST digits from the VE-SDE model using the Predictor only sampler with 50, 100, 250, 500, and 1000 reverse time steps.

### C. Effect of the Number of Sampling Steps

The number of function evaluations (NFE) used by the reverse time SDE sampler is a key computational cost driver at inference time. We swept the number of discretization steps over  $\{50, 100, 250, 500, 1000\}$  and measured wall-clock sampling time, FID, mean classifier confidence, and recognized-digit fraction for each configuration, using 64 samples for the timing/pixel statistics measurements and 1,000 samples for the FID and classifier-based evaluation. The complete results are reported in Table II.

Sampling time scales are approximately 4. As seen in Table II, sampling time scales approximately 4. Increase proportionally to the number of steps, increasing from 0.715 s at 50 steps to 15.922 s at 1000 steps for a batch of 64 images (roughly 0.011 s/sample to 0.249 s/sample). This is in contrast to FID, which does *not* get better as more steps: it decreases from 48.12 at 50 steps to a minimum of **45.11 at 250 steps**, after which it plateaus and then slightly *worsens* at 500 and 1000 steps (45.28 and 47.38, respectively). This behavior does imply that, for this model and dataset, 250 steps are a good compromise since they are beyond what is needed and yet not overwhelming. Further function evaluations provide decreasing or even negative benefits. The distribution quality is not good, possibly because of the accumulation of discretization or numerical, This big error in the reverse-time integration with very small step sizes. The classifier confidence and digit recognition fraction have a broadly similar pattern to each other. A similar trend is observed with the maximum of 250 steps at 0.9305 confidence, 0.9844 digit fraction. before reducing at 500 steps, even though digit fraction is at its nominal From the same FID, a maximum of 1.0 at 1000 steps even with the degradation of FID – suggesting that at very high step counts the classifier finds the digits Despite a distribution produced overall that is less recognizable, each individual is more recognizable Somewhat less accurate with the actual data distribution. Fig. 4 Plots the FID trend as a function of sampling steps.

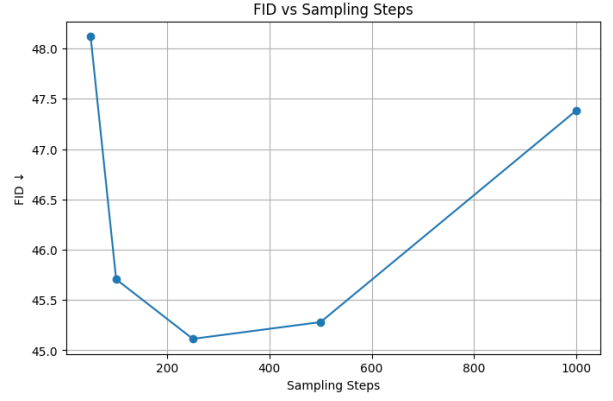


Fig. 4. FID as a function of the number of reverse time sampling steps

TABLE III  
PREDICTOR ONLY VS. PREDICTOR-CORRECTOR (PC) SAMPLING: FID, COST, AND CLASSIFIER BASED QUALITY METRICS.

| Method    | Steps | NFE  | FID ↓          | Time (s) | Conf.  | Digit Fr. |
|-----------|-------|------|----------------|----------|--------|-----------|
| Predictor | 50    | 50   | 48.1189        | 0.71     | 0.8921 | 0.9688    |
| PC        | 50    | 100  | 40.4739        | 21.97    | 0.8165 | 0.9040    |
| Predictor | 100   | 100  | 45.7057        | 1.44     | 0.9170 | 0.9688    |
| PC        | 100   | 200  | 37.0292        | 46.31    | 0.8402 | 0.9310    |
| Predictor | 250   | 250  | 45.1138        | 3.77     | 0.9305 | 0.9844    |
| PC        | 250   | 500  | <b>35.8669</b> | 115.54   | 0.8522 | 0.9370    |
| Predictor | 500   | 500  | 45.2804        | 7.78     | 0.9027 | 0.9688    |
| PC        | 500   | 1000 | 38.5971        | 233.15   | 0.8466 | 0.9220    |
| Predictor | 1000  | 1000 | 47.3823        | 15.92    | 0.9153 | 1.0000    |
| PC        | 1000  | 2000 | 42.6409        | 469.44   | 0.8492 | 0.9290    |

### D. Predictor vs. Predictor Corrector Sampling

We also tested a Predictor-Corrector (PC) sampler, in which each Euler Maru Yamah step one corrector by Langevin dynamics Table III For both the digit fraction and the classifier confidence are reported, along with the sampling time. Sample of samplers in all five step budgets.

Table III indicates that the PC sampler always gets a At each matched *FID* is substantially lower than the Predictor only sampler. step budget. This project obtained the best overall FID as Here is a comparison between the samplers: the PC sampler at 250 steps (PC250) and the The best Predictor only FID of 45.11 at 250 steps (P250); an improvement of **9.25 FID points**. This is to confirm the corrector step in Langevin. correctly corrects for discretization, meaningfully improves distributional fidelity

Despite its *lower* for the PC sampler than for the Predictor-only sampler, the latter is still not as widely used. The two measures captured different aspects of sample quality, with better FID indicating better sample quality. quality: FID is a measure of the goodness of the overall distribution produced. The classifier used a synthetic data distribution that was different from real data distribution in a learned feature space. metrics are more sensitive to the sharpness/legibility of individual digit strokes. The Fig. 5 shows the gap in the FID from the two samplers. In all step budgets.

The PC sampler's FID is *non-monotonic* like that of the Predictor-only sampler. In the number of steps: it is improved

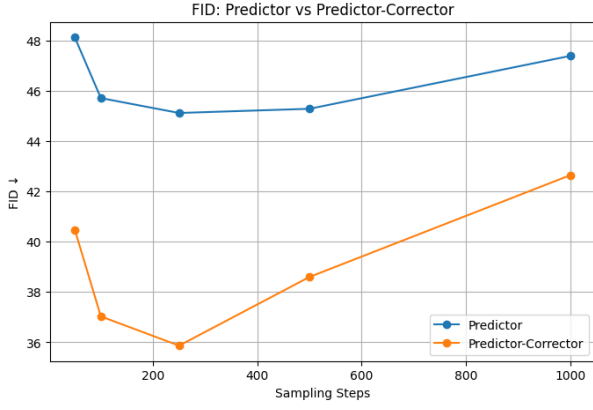


Fig. 5. FID comparison between the Predictor-only and Predictor-Corrector samplers across sampling-step budgets.

TABLE IV  
QUALITY EFFICIENCY TRADE OFF SCORES FOR THE PREDICTOR ONLY SAMPLER (NORMALIZED)

| Steps | FID     | Time (s) | FID Qual. | Eff.   | Trade-off     |
|-------|---------|----------|-----------|--------|---------------|
| 50    | 48.1189 | 0.715    | 0.0000    | 1.0000 | 0.5000        |
| 100   | 45.7057 | 1.441    | 0.8030    | 0.9523 | 0.8776        |
| 250   | 45.1138 | 3.771    | 1.0000    | 0.7990 | <b>0.8995</b> |
| 500   | 45.2804 | 7.784    | 0.9446    | 0.5351 | 0.7399        |
| 1000  | 47.3823 | 15.922   | 0.2451    | 0.0000 | 0.1226        |

from 40.47 (50 steps) to at least 35.87 (250 steps), then degrades again at 500 and 1000 steps (38.60 and 42.64, respectively). This is in keeping with the trend of the Predictor only and strengthens the indication that 250. This is the empirically best discretization budget for reverse time steps. A model that is independent of the sampler variant used.

#### E. Quality Efficiency Trade off

Quantify the cost of generation quality vs computational cost for data samples (NX) to be distributed between 0 and 1. Normalizing the data samples (NX) and the FID (quality) between 0 and 1 was the case for the Predictor only sampler.

The **250-step configuration** is listed in Table IV as such, as it is the most suitable. As shown in Table IV the **250-step configuration** is the most suitable configuration. achieves the highest combined trade-off score (0.8995), jointly maximizing FID, the figure of merit, and F11, the figure of merit for the number of FREs diagnosed. and medium sampling cost, closely followed by the Lower configuration 100-step (0.8776) has nearly the same quality at lower price. There is a significant reduction of about 2.6 $\times$  in sampling time. The 1000 step configuration is sent to the the highest number of events, and the most sophisticated algorithm with the highest number of events and the most sophisticated algorithm, and the lowest trade off score (0.1226). The high sampling latency is not balanced by a matching FID, improvement. These results make clear that simply increasing the number of The reverse time integration steps is *not* an efficient way of enhancement. Once a moderate step budget ( 250) is achieved, generation quality in this setting is one step higher. been reached.

TABLE V  
BEST CONFIGURATIONS IDENTIFIED ACROSS ALL EXPERIMENTS.

| Criterion                         | Configuration         | Value               |
|-----------------------------------|-----------------------|---------------------|
| Best FID (Predictor)              | 250 steps (P250)      | FID = 45.1138       |
| Best FID (overall)                | PC, 250 steps (PC250) | FID = 35.8669       |
| Fastest sampling                  | Predictor, 50 steps   | 0.71 s / 64 samples |
| Best quality/efficiency trade-off | Predictor, 250 steps  | Score = 0.8995      |
| Classifier test accuracy          | Evaluation CNN        | 99.53%              |

#### F. Summary of Key Findings

Table V consolidates the best configurations identified across all experiments.

### VII. DISCUSSION

#### A. Probabilistic Interpretation

The main question here is whether the learned score field is accurate enough to be used for guidance in performing stochastic reverse trajectories for the distribution of the MNIST data. A lower FID means that the feature-space metric has better agreement with the reference distribution, but it does not indicate that the two distributions are identical. There is also room for high confidence in auxiliary classifiers that help make things recognizable, but also allows modes to be missing or diversity lower. A possible interpretation of the PC corrector is that it is a local score-guided MCMC refinement step between predictor updates. This is in favor of the hypothesis that correction enhances the numerical approximation of the reverse-time sampling. If PC yields a small benefit but doubles NFE, the point of its use is not that great. If predictor-only reaches are of comparable quality at higher values of step count, then some correction benefit can be made up for by using additional discretization.

#### B. Quality-Computation Trade-off

Since sampling is sequential, the number of score evaluations will increase roughly linearly as a function of the number of steps. The difference between the evaluations of PC and the one corrector is that the evaluations of the PC are about double those of the one corrector. Therefore, a better comparison is quality per computational budget, and not just on the basis of FID.

#### C. Sampling Stability and Uncertainty

Both forward perturbation and reverse generation are stochastic. Thus, various initial noise realisations may yield different results. If more than one run is performed, the mean and standard deviation should be reported for FID and other stochastic measures. Numerical sensitivity can also be detected from stability within step budgets.

#### D. Failure Cases and Limitations

The structure of the result containers for the predictor experiment and the PC experiment were different such that the code for comparing the results had to be modified. Because MNIST is a small and relatively simple dataset, findings for this dataset do not necessarily apply to larger, higher-resolution

distributions. Moreover, FID is a feature space metric that isn't an exact likelihood metric, but is instead one that applies to a finite number of samples. Another limitation in the present PC configuration was that only one corrector step was tested with an SNR of 0.16. Lastly, wall-clock sampling time depends on the Colab runtime environment used and should be taken at the interpretation of computational efficiency for a specific environment rather than as a generic measure.

#### E. Threats to Validity

Important threats include limited dataset complexity, finite generated-sample counts, runtime variability, and sensitivity to sampler hyperparameters. Holding the trained score model fixed reduces confounding between the two sampling methods, but repeated runs and broader corrector sensitivity experiments would strengthen the conclusions.

### VIII. CONCLUSION

This work used a time-conditioned U-Net for the MNIST task and a VE-SDE score-based generative model and examined if the quality of the samples generated by the Predictor–Corrector sampling method is sufficient to warrant extra computational power. It is important to include a meaningful baseline that employs predictor-only reverse-SDE sampling, as this type of reverse uses the same score model and reverse dynamics. PC introduces just a Langevin refinement, separating out the effect of correction. Final answer to be drawn from completed FID, Classifier, Timing, NFE, and Qualitative comparisons. The criterion is based on whether PC has a significant improvement in the quality of the product while using the same level of computing resources, or using fewer actual steps to achieve the same quality of product. The more general probabilistic perspective is a balance between the accuracy of the numerical reverse-time inference and the cost of stochastic sampling. Repeated sampling runs, an in-depth sensitivity analysis over corrector SNR and corrector-step count, larger datasets, and improved numerical solvers are all recommended for future work.

### IX. TEAM CONTRIBUTION STATEMENT

TABLE VI  
TEAM CONTRIBUTION

| Member                      | Contribution                                                                                                     |
|-----------------------------|------------------------------------------------------------------------------------------------------------------|
| Tasnia Mahjabin Mal-<br>iha | Probabilistic formulation; theory; literature review; evaluation; visualization; analysis; writing and revision. |
| Sumaiya Azad Isha           | U-Net implementation; training pipeline; debugging; predictor and PC sampler implementation; experiments.        |

### REFERENCES

- [1] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-Based Generative Modeling through Stochastic Differential Equations,” arXiv:2011.13456 [cs, stat], Feb. 2021. [Online]. Available: <https://arxiv.org/abs/2011.13456>.
- [2] Y. LeCun, C. Cortes, and C. J. C. Burges, “MNIST handwritten digit database,” 1998. [Online]. Available: <http://yann.lecun.com/exdb/mnist/>.
- [3] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. [Online]. Available: <https://arxiv.org/abs/1912.01703>.
- [4] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and G. Hochreiter, “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.08500>.